Lab Two: File Management and Filesystem Navigation in RHEL 9

Section 1: Understanding the Filesystem Hierarchy

1. What command can you use to view the filesystem hierarchy from the root directory (/)?

Answer: ls -R /

2. What is the purpose of the /home, /var, and /usr directories in the Linux filesystem hierarchy?

Answer: /home: Contains user home directories

/var: Stores variable data

/usr: Contains user utilities and applications

Section 2: Identifying File Types in Linux

3. What command do you use to determine the type of a file in Linux?

Answer: file filename

4. What are the main types of files available in Linux?

Answer: Regular **files** (-): Text, binary, images , character files , block files , special files are directories which points to files

Directory (d): A folder containing files or other directories.

Section 3: Navigating Directories with Absolute and Relative Paths

5. How do you navigate to a directory using an **absolute path**?

Answer:
```
abdelhamed@abdel:~$ cd /boot/grup
```

6. How do you navigate to a directory using a **relative path**?

Answer:
```
abdelhamed@abdel:~$ cd snap/
```

7. What is the difference between absolute and relative paths in Linux?

Answer:

- **absolutes Path:** Starts from the root / and gives the full path.
- **Relative Path:** Starts from the current working directory.

Section 4: Listing Directory Contents

8. What command do you use to list the contents of a directory?

Answer:
```
abdelhamed@abdel:/$ ls home/
abdelhamed
```

9. How do you list **hidden files** in a directory?

Answer:
```
abdelhamed@abdel:/$ ls -a home/
.  ..  abdelhamed
abdelhamed@abdel:/$
```

Section 5: Creating, Copying, Moving, and Removing Files and Directorie?

10. How do you create a new directory in the shell?

Answer :
```
abdelhamed@abdel:~/Desktop$ mkdir Assignments
```

11. What is the command to create an empty file?

Answer:
```
abdelhamed@abdel:~/Desktop/Assignments$ touch AsignemntOne.txt
abdelhamed@abdel:~/Desktop/Assignments$
```

12. How do you **copy** a file in the shell?

Answer :

```
abdelhamed@abdel:~/Desktop/Assignments$ cp AsignemntOne.txt ../
abdelhamed@abdel:~/Desktop/Assignments$ ls ../
AsignemntOne.txt   aws      CAN_Socket.c  ETC_PASSWD
Assignments        CAN.elf  CAN_SOk.c
```

13. How do you **move** a file in the shell?

Answer:

```
abdelhamed@abdel:~/Desktop/Assignments$ mv AsignemntOne.txt ../
abdelhamed@abdel:~/Desktop/Assignments$ ls
abdelhamed@abdel:~/Desktop/Assignments$
```

14. How do you **remove** a file or directory?

Answer: 1- For file, we use **rm**

For Directories, if contain files use **-> rm -rf**

Else **-> use rmdir**

Section 6: Creating Hard Links and Soft Links

15. What is the difference between a hard link and a soft link (symbolic link)?

Answer :

1 – Hard link : is a link to inode of the file , used in the backup of the files , without duplicate the size of the file , it cannot be linked to folders , when u delete the main file , the linked file won't be affected as if pointed to the inode number every thing is found in the linked

2 – Soft Link: link to the original file path, size depend on the length of the path of the file text , if the original file changed its name or deleted ,, the linked file will be corrupted , it has arrow, u can navigate through ls -l and look out

16. How do you create a **hard link** in Linux?

Answer: **ln original.txt hardlink.txt**

17. How do you create a **soft (symbolic) link** in Linux?

Answer**: ln -s original.txt symlink.txt**

18. What happens if you delete the original file (example.txt) while the soft link still exists? What about the hard link?

Soft Link: Becomes **broken**. It points to a non-existent file.

```
abdelhamed@abdel:~/Desktop/Assignments$ ls -ll
total 4
lrwxrwxrwx 1 abdelhamed abdelhamed 16 Nov 29 19:16 Linked.txt -> originalfile.txt
-rw-rw-r-- 1 abdelhamed abdelhamed 12 Nov 29 19:16 originalfile.txt
abdelhamed@abdel:~/Desktop/Assignments$ rm originalfile.txt
abdelhamed@abdel:~/Desktop/Assignments$ ls -ll
total 0
lrwxrwxrwx 1 abdelhamed abdelhamed 16 Nov 29 19:16 Linked.txt -> originalfile.txt
abdelhamed@abdel:~/Desktop/Assignments$
```

Hard Link: Still works. The data remains accessible because the inode still exists.

```
abdelhamed@abdel:~/Desktop/Assignments$ touch OriginalFile.txt
abdelhamed@abdel:~/Desktop/Assignments$ ln OriginalFile.txt HardLinkedFile.txt
abdelhamed@abdel:~/Desktop/Assignments$ echo "Hello Hard Link" >> OriginalFile.txt
abdelhamed@abdel:~/Desktop/Assignments$ cat OriginalFile.txt
Hello Hard Link
abdelhamed@abdel:~/Desktop/Assignments$ cat HardLinkedFile.txt
Hello Hard Link
abdelhamed@abdel:~/Desktop/Assignments$ rm OriginalFile.txt
abdelhamed@abdel:~/Desktop/Assignments$ cat HardLinkedFile.txt
Hello Hard Link
abdelhamed@abdel:~/Desktop/Assignments$
```